

Final Project: An AI Security Analyst for Your Own Lab

Build your own scanning tools, put brakes on them, and build an assistant that turns their output into a grounded report.

A two person capstone • 200 points

1. The idea

You build a lab, write your own scanning tools to probe it, and build an assistant that explains what they found. You build the assistant three ways, the same arc as the module notebooks, and then say which is worth what.

Stage	What it does	What you learn from it
Chain	the model writes the report from its own memory	where it guesses, and how confidently
Retrieval	every finding is grounded in a real advisory	how much the knowledge base decides the answer
Agent with MCP	the agent picks tools and decides the steps	what autonomy buys, and what it costs

An agent is only as good as the tools it can reach, so you build the toolbox: small Python scanners that mimic the recon side of a tool like Metasploit. A gap you find, explain, and close is worth more than a clean run.

Why the brakes, for security

The moment an AI can run security tools on its own, the danger is no longer a bad answer, it is a bad action against the wrong system. Professional automation is defined by its brakes: a scope check enforced in code, and a human who approves every action. You build the engine and the brakes together.

2. Who does what

	Leads	Target	Knowledge base
Partner A	the service side	Metasploitable 2, or a vulnerable service image	cve_knowledge_base.md
Partner B	the web side	OWASP Juice Shop, DVWA, or WebGoat	web_security_kb.md
Together	the integration	one agent over both toolsets	both

Both of you build all three stages and your own tools. Teams of three need approval and a wider scope.

3. Before you start

- **Lab, and where it runs.** Metasploitable 2 is a full virtual machine: run it on your own computer in VirtualBox or VMware, on a host only network (it is x86, so on Apple silicon use Docker instead). Docker targets, Juice

Shop, DVWA, WebGoat, or a vulnerable service image, come up in one command on your own machine or on your Jetstream2 instance. Jetstream2 cannot host the VM; its job here is the model below.

- **Tools.** You write the scanners in Python, with `python-nmap` (or `nmap` from Python) and `socket`, `ftplib`, `urllib`, and `ssl` from the standard library. The starter server shows a `port_scan` and a commented `nmap` call to build from; replace the simulated scans with real ones against your lab.
- **Two models.** Host one on your Jetstream2 instance, where the GPU carries a fourteen billion parameter class model your laptop cannot, and use a paid API for the other. If Jetstream2 is unavailable, host the largest model your hardware or a free cloud GPU will carry, say which, and compare that. No credit is lost.
- **Start from** `part2_orchestration_simulated` and `part3_web_pentest_real` and their servers.

Caution: scope, legality, and safety, read first

Test only the lab you built. Put a virtual machine on a host only adapter, publish container ports to localhost only (`-p 127.0.0.1:3000:3000` , not to every interface), and on Jetstream2 leave the firewall closed. Your tools also enforce this in code, through the scope allowlist in §6. Keep API keys in `.env` or Colab Secrets, never in the repository, and cite every advisory in `SOURCES.md` .

4. The task list

	Task	Who	Done when
1	Build the lab and get a first scan	each, own target	full tool output in the notebook, not a summary
2	Bring up both models	together	the same prompt runs on the hosted model and on the API
3	Stage 1, the chain report	each partner	you can point at a line where it guessed
4	Stage 2, retrieval and coverage check	each partner	every finding is sorted into grounded or gap
5	Build your scanning toolkit	each partner	three or more Python scanners over MCP, each returning a real finding
6	Add the guardrails (§6)	together	tools refuse out-of-scope targets in code; the agent waits for typed approval
7	Ground everything the tools find	each, own base	every finding type has a cited entry with severity and fix
8	Stage 3, one agent over both sides	together	it chooses its tools, obeys the guardrails, and grounds each finding
9	Push coverage as far as it goes	together	one report covers both sides, and the gaps that remain are named
10	Compare the two models	together	same findings through both, with accuracy, time, cost, and exposure

5. Your scanning toolkit

the heart of the project

At least three scanners per side, each a Python function wrapped as an MCP tool that mimics a recon or enumeration module of a tool like Metasploit. They detect and enumerate; they do not run exploits. Some ideas:

Scanner	What it checks	Python to reach for
service and version scan	open ports and the software behind them	<code>python-nmap</code> (<code>nmap -sV</code>)

Scanner	What it checks	Python to reach for
banner grab	what a service announces on connect	<code>socket</code>
anonymous FTP check	a login that needs no password	<code>ftplib</code>
SMB share enumeration	shares, and which are readable or writable	<code>smbprotocol</code> , or <code>smbclient</code> from Python
HTTP method or TLS probe	dangerous verbs, or weak ciphers	<code>urllib</code> , <code>ssl</code>

Each scanner produces a new kind of finding that needs grounding, so your knowledge base grows with your toolkit (a second file is fine). Give every tool a docstring precise enough for the agent to pick it.

6. Guardrails

Because your tools run against a real machine, the system needs two brakes, both visible in the notebook:

- **Scope, in code, fail closed.** Define your lab's addresses once as an allowlist; every tool checks the target on its first line and refuses anything else, so a scan can never reach a host you do not own.
- **A human gate.** The agent proposes its next action and pauses; the follow-up runs only when a person types `APPROVE`. Nothing acts on its own.

Security lesson: verify the AI

The analyst's triage is a recommendation, not a decision. Two things to check yourself: did any finding cite a CVE the model may have invented rather than looked up, and is it truly impossible for an action to run without your typed approval? Confident but unverified output and an agent that acts on its own are the two biggest risks in AI assisted security work; your allowlist and your gate exist to contain them.

7. What to hand in

Submit this

One team report (PDF), with your GitHub folder link, your video link, and both names on page one. Everything else lives in the repository.

Repository. Add a `final_project/` folder to your private course repository, with your instructor and the TA as collaborators, holding: a `README.md`, the run notebook with outputs saved, your `recon_server.py`, the two knowledge bases, `SOURCES.md`, a `results/` folder, and a `CONTRIBUTIONS.md` signed by both partners.

Report, written together:

1. Scope, and a findings table across both sides, each finding with its CVE, CWE, or OWASP category and evidence.
2. Your toolkit and its brakes: the scanners you wrote and the Metasploit function each echoes, the allowlist, and the human gate.
3. Knowledge base: what you added to ground your tools' findings, and coverage before and after.
4. Chain against retrieval against agent: groundedness, effort, scaling, autonomy, failure modes.
5. Hosted model against the API: accuracy, time, cost, and where your data went.
6. Where each earlier module shows up in what you built, and a short reflection on the ethics of automating it.

Video, twelve to eighteen minutes, both partners speaking:

- each presents the side they led, the scanners they wrote, and their coverage numbers;
- a live run of the agent driving your tools, including a refused out-of-scope target and the human gate;
- each partner answers a question about the other's side.

8. Grading

Each partner earns the points for the side they led; both share the team points (adjusted if the contributions statement and commits show an uneven split).

	Criterion	Points
Team	Lab built, both models running, real scan output carried through the arc	15
Each partner	Your side: three stages, three or more scanners over MCP, and your knowledge base grown and cited to ground them, with coverage before and after	35 each
Team	Guardrails: a scope allowlist enforced in code and fail closed, and a human gate before any action	20
Team	Integration: one agent over both toolsets and both knowledge bases, choosing its tools	20
Team	Coverage: broad, correct, each finding mapped, remaining gaps named	15
Team	The three approaches compared, from your own numbers	20
Team	The two models compared, including cost and data exposure	15
Team	Video: both present, live demo works, each speaks to the other's side	15
Team	Repository, contributions statement, a reflection from each partner	10
Total		200

Caution: automatic fail

Scanning any target outside your allowlist fails the project. A committed API key loses 10 points.